

Contratto Claim Keycloak — Integrazione autenticazione Cruscotto

Documento operativo per l'**amministratore Keycloak**. Definisce il client da creare e i **claim** che il token di accesso (access token) deve contenere per integrarsi correttamente con il cruscotto.

1. Client Keycloak da configurare

Il frontend è una SPA (Single Page App) e usa il flusso **Authorization Code + PKCE**.

Parametro	Valore
Tipo client	Public (SPA, nessun client secret)
Standard Flow (Authorization Code)	ON
PKCE	S256 (obbligatorio)
Direct Access Grants	OFF
Valid Redirect URIs	URL dell'app (es. <code>https://cruscotto.<dominio>/*</code>)
Valid Post Logout Redirect URIs	<code>https://cruscotto.<dominio>/</code>
Web Origins	<code>https://cruscotto.<dominio></code> (o +)

Il frontend si configura con 3 variabili d'ambiente (lato app, non Keycloak): `VITE_KEYCLOAK_URL`, `VITE_KEYCLOAK_REALM`, `VITE_KEYCLOAK_CLIENT_ID`.

2. Claim richiesti nel token di accesso

2.1 Ruolo utente — obbligatorio (già standard in Keycloak)

L'app legge i ruoli da: - `realm_access.roles` (ruoli di realm), **e/o** - `resource_access.<CLIENT_ID>.roles` (ruoli di client).

Regola di mapping applicativa (case-insensitive): se l'utente ha **uno** di questi ruoli è trattato come **amministratore** (profilo dfp, vede tutti gli enti):

dfp, super_admin, superadmin, admin, amministratore, amministratore-gru, amministratore-formez, amministratore-unico

Qualsiasi altro utente è trattato come **operatore di ente** (profilo `ente_hr`, vede solo i propri enti). ⇒ È sufficiente assegnare all'utente uno di questi ruoli (per gli admin) oppure un ruolo "ente"/nessuno di quelli sopra (per gli operatori).

Nessuna configurazione extra: i ruoli sono già inclusi nel token da Keycloak.

2.2 Enti dell'utente — obbligatorio per gli operatori (`ente_hr`)

L'identità dell'ente **DEVE** essere il **CODICE FISCALE dell'ente** (chiave stabile con cui il sistema collega tutti i dati). **Non** usare id interni, né il codice ISTAT del comune.

Aggiungere **un** claim, a scelta secondo lo scenario:

Scenario	Claim	Tipo	Esempio
Utente legato a 1 solo ente	cf	stringa	"80078750587"
Utente legato a più enti	enti	array di stringhe	["80078750587", "00514490010"]

- Formato CF ente: stringa di **11 cifre** (codice fiscale dell'ente).
- Per gli utenti **amministratori** (dfp) il claim ente **non serve** (vedono tutto).
- Deve coincidere con `dwh.lk_questionari_enti.codice_fiscale_ente` / `ca.lk_istituzioni.CODI_FISCALE` del data warehouse.

2.3 Dati anagrafici utente — standard OIDC (consigliati)

Già presenti se abilitati gli scope `profile/email`: - `name` (o `preferred_username`) → nome mostrato nell'interfaccia. - `sub` → identificativo stabile dell'utente (utile per collegarlo al DB login). - `email` (facoltativo).

3. Come aggiungere il claim ente (mapper Keycloak)

1. Salvare il CF come **attributo utente**: es. attributo `cf` (o `enti`) nella scheda utente (o via User Federation/sincronizzazione dalla tabella `login_enti`).
2. Client → **Client scopes** → (scope dedicato o dedicated scope del client) → **Mappers** → *Add mapper* → **User Attribute**: - Name: `cf` - User Attribute: `cf` - Token Claim Name: `cf` - Claim JSON Type: String (per `enti`: String + **Multivalued ON** → array) - **Add to access token: ON** (fondamentale: l'app legge l'**access token**) - Add to ID token: ON (facoltativo)
3. (Opz.) Per gli admin nessun mapper ente necessario.

4. Esempio di access token (payload decodificato)

Operatore di ente (mono-ente):

```
{
  "sub": "f7c1...-user",
  "preferred_username": "mrossi",
  "name": "Mario Rossi",
  "realm_access": { "roles": ["ente_hr"] },
  "resource_access": { "cruscotto": { "roles": [] } },
  "cf": "80078750587"
}
```

Operatore multi-ente:

```
{
  "name": "Anna Bianchi",
  "realm_access": { "roles": ["ente_hr"] },
  "enti": ["80078750587", "00514490010"]
}
```

Amministratore (DFP/Formez):

```
{
  "name": "Utente DFP",
  "realm_access": { "roles": ["amministratore-formez"] }
}
```

5. Riepilogo: claim → uso nell'app

Claim	Obbligatorio	Uso
realm_access.roles / resource_access.<client>.roles	Sì	Determina ruolo dfp vs ente_hr
cf oppure enti	Sì per ente_hr	CF ente/i → filtro dati sui soli enti dell'utente
name / preferred_username	Consigliato	Nome mostrato
sub	Consigliato	Aggancio all'utente nel DB (login)
email	Facoltativo	—

6. Nota di allineamento lato app

Oggi il codice legge provvisoriamente un claim `istatcode` interpretandolo come id interno dell'ente. Con l'adozione di questo contratto (`cf/enti` = codice fiscale) è previsto un **piccolo adeguamento** di `src/contexts/AuthContext.tsx`: leggere `cf/enti` e risolvere CF → id interno tramite la vista `dw_ente`. Modifica minima e retro-compatibile.

In sintesi, all'amministratore Keycloak va chiesto di: 1. creare il client SPA (public + PKCE S256) con i redirect URI corretti; 2. assegnare i **ruoli** (uno dei ruoli "amministratore" per i DFP; ruolo ente/nessuno per gli operatori); 3. aggiungere al **access token** il claim `cf` (o `enti` se multi-ente) = **codice fiscale dell'ente**.